#hackthekitty 2026

Project Report

Project Name: Nine Lives

Reference ID: 54EOUNWX

---

1. Executive Summary

Nine Lives is a roguelike web game where users upload a photo of their cat, which is digitized through an ML/AI pipeline into a unique playable battle character with generated stats, abilities, and lore. When a cat exhausts all nine lives, it is laid to rest in a Memorial — an interactive space dedicated to fallen companions, inspired by the real experience of losing a pet.

2. Project Overview

2a. Why you're building what you're building

Most games generate characters from templates or random seeds. Nine Lives starts from something real — a photo of your actual cat. The motivation came from a personal place: the idea of honoring a pet that has passed by giving them a second life in a game world. The Memorial mechanic turns what is usually a "game over" screen into something emotionally meaningful: a place to revisit, reflect, and remember.

2b. How it relates to the theme

Cats are not just the theme — they are the entire game. The player's cat is the protagonist, generated from a real photo. The enemies are procedurally generated cats. The Memorial is a space for fallen cats. Every mechanic, from the 9-lives system to the breed-based stat generation, is built around cats as the central concept.

2c. Target Audience

- Cat owners who want to see their pet become a game character
- Casual roguelike players looking for a short, replayable experience
- Anyone who has lost a pet and would find meaning in a memorial mechanic

Hackathon judges evaluating creative use of ML/AI in a product context

3. Key Features

- **Cat Digitizer** — upload a photo of your cat; the ML pipeline identifies the breed, extracts dominant fur colors, generates a full battle card (name, class, stats, 4 abilities, lore) via LLM, and produces a stylized pixel-art avatar
- **Turn-based combat** — fight procedurally generated cat enemies with a mana/cooldown ability system; difficulty scales with each round
- **9-Lives system** — your cat revives on death until all lives are exhausted, giving each run genuine stakes

- **Memorial Space** — fallen cats are preserved in an interactive memorial where you can revisit them and add personal notes
- **Overworld hub** — between battles, a location map lets you choose where to go next, adding a roguelite navigation layer

4. Technology Stack

List the languages, frameworks, libraries, and platforms used. Layer is something like (frontend, backend, database, etc)

Layer	Technology
Frontend	React 19 + Vite + TypeScript
Styling	Tailwind CSS v4 + 8bitcn component registry + framer-motion
Font	Press Start 2P (pixel display font)
Backend	Python FastAPI
Database + Auth	Supabase (PostgreSQL + Row Level Security)
ML — breed classification	HuggingFace transformers (local ViT: dima806/ cat_breed_image_detection)
ML — color extraction	YOLO11s-seg (ultralytics) + scikit-learn KMeans + OpenCV
LLM — card generation	Gemini 2.5 Flash (`google-genai`)
Image generation	FLUX.1-schnell via HuggingFace Inference Providers
Deployment	Vercel
Security	Supabase RLS + JWT auth on all backend endpoints + Aikido security scan

5. Technical Architecture

Component overview

The system is split into three layers: a React frontend (rendering and input only), a FastAPI backend (all game logic, ML pipeline, and data access), and Supabase (database and auth).

Key architectural principle: The frontend has no direct database access. It uses the Supabase client only for authentication (login, session, JWT). All data reads and writes go through authenticated backend endpoints. The backend is the sole authority on game state.

Digitization pipeline

User uploads photo

→ FastAPI /api/digitize

→ HuggingFace ViT (local) → breed name

→ YOLO segmentation + KMeans → dominant fur colors + ratios

→ Gemini 2.5 Flash → name, class, stats, 4 abilities, lore, image prompt

→ FLUX.1-schnell (fal.ai) → pixel-art avatar image

→ Supabase → cat record persisted

Battle flow

Frontend → POST /api/battle/start → initial GameState returned

Frontend → POST /api/battle/action → backend resolves full turn (player + enemy)

→ state persisted to game_run.state (JSONB)

→ updated GameState returned to frontend

Data model (simplified)

- `auth.users` → `cat` (1:many) → `game_run` (1:many)
- `game_run.state` (JSONB) holds full mid-run battle state
- `cat.status` = ALIVE | MEMORIAL
- Enemy is procedurally generated server-side, never persisted

Security

- Every backend endpoint requires a Supabase JWT
- Ownership of `cat` and `game_run` verified against authenticated user before any operation
- RLS policies as defense-in-depth at the database level
- Aikido security scan passed pre-submission
- API keys stored in environment variables only, never exposed to the frontend

6. Testing Matrix

Optional section, but this will improve your marks on technical execution. List the manual test cases you ran to verify your project works as intended.

Feature / Flow	Steps	Expected Result	Actual Result	Pass / Fail
Image upload validation	Upload a .pdf file	Error message, upload rejected	Error displayed	Pass
Image upload validation	Upload a >10MB image	Error message, upload rejected	Error displayed	Pass
Cat digitization	Upload valid cat photo, submit	Breed detected, card generated, avatar created, navigated to battle	Full card generated	Pass
Battle initialization	Navigate to battle after digitize	Initial GameState loaded, enemy generated for round 1	State loaded correctly	Pass
Basic attack	Click Attack	Enemy HP decreases, enemy takes turn	HP reduced as expected	Pass
Defend action	Click Defend, receive enemy attack	Damage reduced by 50%	Damage halved	Pass
Ability usage (mana check)	Use ability with insufficient mana	Action rejected, error shown	Rejected correctly	Pass
Ability cooldown	Use ability, attempt again immediately	Ability unavailable until cooldown expires	Cooldown enforced	Pass
9-lives revival	Let HP reach 0 with lives remaining	Cat revives at full HP, life counter decrements	Revival triggered	Pass

Feature / Flow	Steps	Expected Result	Actual Result	Pass / Fail
Game over	Exhaust all 9 lives	Cat moved to Memorial, run marked COMPLETED	Memorial updated	Pass
Round progression	Defeat enemy	Round increments, new harder enemy generated	New enemy appears	Pass
Memorial display	Navigate to Memorial	All fallen cats displayed with stats and death date	Cats listed	Pass
Auth protection	Access /battle without login	Redirected to login	Redirect triggered	Pass
Cross-user access	Submit action for another user's run	403 Forbidden returned	403 returned	Pass
State persistence	Refresh browser mid-battle	Game resumes from last saved state	State restored	Pass

8. Future Improvements

- **Inventory system** — items dropped by defeated enemies that grant stat bonuses, stored on the cat record
- **Damage type resistances** — breed-based elemental affinities (e.g. fire resistance for flame-colored cats)
- **Overworld movement** — free movement through the world map rather than node selection, with the camera following the cat
- **Interactive Memorial** — cats placed spatially in a memorial garden you can walk through, not just a list
- **Attribute scaling** — STR/AGI/INT stats that gate specific ability types and scale damage formulas
- **Multi-cat runs** — bring a team of cats, switch between them mid-battle
- **Social sharing** — share your cat's memorial card as an image

9. Tools You Used

- **Kiro IDE** — primary AI coding assistant (agentic mode, used for implementation)
- **OpenCode** — secondary AI coding assistant
- **Claude (Anthropic)** — architecture planning, prompt engineering, documentation
- **ElevenLabs** — AI voiceover for demo video
- **OBS Studio** — screen recording for demo
- **Supabase Dashboard** — database management and RLS configuration
- **Aikido Security** — automated security scan
- **freesound.org** — ambient music and sound effects

11. Learnings & Takeaways

Technical:

- Building a full ML pipeline (classification → segmentation → color extraction → LLM → image gen) as a single cohesive product flow is harder to orchestrate than any individual step — the handoffs between services are where bugs hide
- YOLO instance segmentation was necessary for reliable cat isolation; simpler background removal tools (rembg, SAM2) failed for different reasons in this environment
- Keeping the frontend as a pure rendering layer with no game logic made debugging dramatically easier — all state issues were traceable to a single backend source of truth
- Scoping a roguelike MVP honestly is an art — the first instinct is always to build more than time allows

Personal:

- The emotional core of a project (in this case, the Memorial) is worth protecting even when time pressure pushes toward cutting it — it's what makes the project memorable to judges and to yourself
- Shipping something imperfect on time is better than not shipping something perfect

12. Acknowledgments

- `dima806/cat_breed_image_detection` (HuggingFace) — fine-tuned ViT for cat breed classification
- `ultralytics/ultralytics` — YOLO11 segmentation model
- `8bitcn` — retro 8-bit component registry for shadcn/ui
- Oxford-IIIT Pet Dataset — foundational dataset for breed classification research
- `black-forest-labs/FLUX.1-schnell` — avatar image generation model
- Google Gemini 2.5 Flash — character card and lore generation

Submission Checklist

- ☐ Video demo (HD or at least 720p)
- ☐ README.md (prerequisites, run instructions, configuration)
- ☐ Project report (this document, in documentation/)
- ☐ Source code in src/
- ☐ No unrelated files, executables, auto-generated code, or package folders (e.g. node_modules/, build/, dist/, vendor/)